Knowledge Bytes

Legacy E-Commerce (Yantra Bazaar) — guided codebase walkthrough

Project root: Legacy E-Commerce · Date: 2026-10-13 · Stack: ASP.NET MVC 5 / EF6 / SQL Server / Razor / jQuery / Docker / Nginx

Each section below follows the Knowledge Bytes format: one concept per byte, plain language first, then the snippet, then mechanics and rationale.

1 Project overview

Yantra Bazaar is a server-rendered demo e-commerce store: product catalog with categories, search, filtering, sorting, pagination and infinite scroll; a session-backed shopping cart with an AJAX mini-cart; ASP.NET Identity registration/login with order history; a multi-step demo checkout (no real payments); and an opt-in Admin area for stock/visibility plus a sales dashboard. It is a classic .NET Framework 4.7 / ASP.NET MVC 5 / Entity Framework 6 application backed by SQL Server, fronted by Nginx, and containerised so it builds with Mono inside Docker — no local .NET install is required. Port 8085 on the host maps to Nginx, which proxies to the XSP4-hosted web app; SQL Server and a one-shot db-init container complete the compose stack.

How the solution is organised

- Ecommerce.Core — plain entities, view models, result wrappers, service interfaces (no I/O).
- Ecommerce.Data — EF6 DbContext, generic repository + unit of work, database health probe.
- Ecommerce.Services — business logic: catalog, cart, checkout/account, admin, plus mapping and pricing rules.
- Ecommerce.Web — MVC 5 controllers, Razor views, Identity auth, Unity DI, static assets (Content/Scripts/Images).
- db/ — idempotent schema.sql, deterministic seed generators (*.py) and generated seed_*.sql, init.sh.
- docker/, nginx/, docker-compose.yml — build, entrypoint, reverse proxy, orchestration.
- tests/ — PowerShell smoke/feature/extended scripts that exercise the running site over HTTP.
- docs/ — operator notes (start/stop commands).

2 Codebase / file map

Relative paths from the project root. Binary image assets under Ecommerce.Web/Images/products/ (~250 category/product photos plus category icons) and generated seed SQL data files are listed as groups rather than individually.

Area	Files	Role
Solution	LegacyEcommerce.sln	Declares the 4 C# projects and build configurations.
Core	Ecommerce.Core/Entities/Entities.cs	9 domain entities (Category, Product, images, variants, Customer, CartItem, Order, OrderLine, Address).
Core	Ecommerce.Core/Interfaces/Interfaces.cs	Repository, unit-of-work and 6 service/health contracts.
Core	Ecommerce.Core/Common/Common.cs	PagedResult<T>, OperationResult / OperationResult<T>.
Core	Ecommerce.Core/ViewModels/*.cs (2 files)	Storefront + admin view models with validation attributes.
Data	Ecommerce.Data/EcommerceDbContext.cs	EF6 context + fluent mappings for all 9 tables.
Data	Ecommerce.Data/Repositories/Repositories.cs	EfRepository<T>, EfUnitOfWork.
Data	Ecommerce.Data/DatabaseHealthCheck.cs	DB connectivity probe for /health.
Services	Ecommerce.Services/Mapping.cs	ToCard mapper + PricingRules (shipping, tax, effective price).
Services	Ecommerce.Services/Impl/CatalogService.cs	Search, detail, category tree/page, featured/new/discounted.
Services	Ecommerce.Services/Impl/CartService.cs	Session cart CRUD, stock checks, totals.
Services	Ecommerce.Services/Impl/CheckoutAccountServices.cs	CheckoutService + AccountService.
Services	Ecommerce.Services/Impl/AdminService.cs	Dashboard metrics, stock/active toggles, IsAdmin check.

Web composition	Global.asax(.cs), Startup.cs, App_Start/*.cs (4), Infrastructure/*.cs (3), Models/IdentityModels.cs	Routes, bundles, filters, Unity DI, OWIN auth, session cart id, admin bootstrap.
Controllers	Ecommerce.Web/Controllers/*.cs (6 files)	Home, Catalog, Cart, Checkout, Account, Admin.
Views	Ecommerce.Web/Views/**/*.cshtml (~30 files)	Layout, header/footer, catalog, product, cart, checkout, account, admin, static pages, partials.
Client	Scripts/site.js, Content/site.css	jQuery interactions; design-system stylesheet (yb-* hooks).
Database	db/schema.sql, init.sh, gen_seed.py, gen_seed1500.py, seed*.sql, seed_product_images.sql	Idempotent schema + deterministic demo catalog data.
Ops	docker-compose.yml, docker/Dockerfile.web, docker/web-entrypoint.sh, nginx/nginx.conf, Web.config, .env.example	Containers, proxy, connection-string injection, config template (no secrets).
Tests/docs	tests/smoke.ps1, features.ps1, extended.ps1, retest.sh; docs/start_and_stopping.txt	HTTP-level verification scripts; operator runbook.
Excluded	_backup_*/ (8 dirs), .git/, bin/, obj/, packages/, Images (binaries), .env (secrets)	Not documented line-by-line; purpose noted in coverage summary.

3 Core domain — Ecommerce.Core/Entities/Entities.cs

This is a C# domain-model file: nine plain entity classes that mirror the nine database tables. Assume the reader knows basic programming but is new to Entity Framework.

Byte 1: The catalogue pair: Category and Product

Builds on: None — starting point

In plain terms:

Categories organise products like departments and aisles in a shop; each product belongs to exactly one category. Slugs are URL-friendly names (for example “men-fashion”) used in pretty routes such as /c/men-fashion.

The code:

```csharp
public class Category {
    public int CategoryId { get; set; }
    public string Slug { get; set; }
    public int? ParentCategoryId { get; set; }
    public virtual ICollection<Category> ChildCategories { get; set; }
    public virtual ICollection<Product> Products { get; set; }
}
```

What's happening: Category carries a nullable ParentCategoryId plus navigation properties (ParentCategory, ChildCategories, Products). virtual enables EF lazy loading; the constructor initialises collections so they are never null. Product holds pricing (Price, optional CompareAtPrice/DiscountPercent), StockQuantity, flags (IsFeatured, IsNewArrival, IsActive), CreatedUtc, and collections of Images and Variants.

Why it matters: This is the vocabulary every other layer shares. Parent/child categories drive the nav tree; IsActive gates storefront visibility; CompareAtPrice powers “save X%” badges.

Byte 2: Product presentation helpers: images and variants

Builds on: Byte 1

In plain terms:

One product can have several photos and several options such as size. These two small classes keep photos and options as separate rows linked back to the product.

The code:

```csharp
public class ProductVariant {
    public int ProductId { get; set; }
    public string VariantName { get; set; } // e.g. Size
    public string VariantValue { get; set; } // e.g. XL
    public decimal? PriceAdjustment { get; set; }
    public int StockQuantity { get; set; }
}
```

What's happening: ProductImage stores ImageUrl/AltText/DisplayOrder per photo. ProductVariant stores a name/value pair (Size/XL), an optional SKU and PriceAdjustment added to the base price, and its own StockQuantity. Both cascade-delete with the product (see DbContext bytes).

Why it matters: Variants let one T-shirt come in four sizes with independent stock; DisplayOrder controls gallery sequence with a placeholder fallback when no image exists.

Byte 3: People: Customer and Address

Builds on: Byte 1

In plain terms:

Shoppers have a lightweight profile linked to their login, plus reusable delivery addresses. Guest checkout is allowed, so these links are optional.

The code:

```
csharp
public class Customer {
    public string AspNetUserId { get; set; }
    public string Email { get; set; }
    public virtual ICollection<Order> Orders { get; set; }
}
```

What's happening: Customer bridges ASP.NET Identity (AspNetUserId) and the shop domain (Email, names, Orders, Addresses). Address holds label, recipient, street/city/state/postal/country, phone and IsDefault. Foreign keys are nullable so guests and address-less rows are legal.

Why it matters: Identity owns passwords; Customer owns purchase history. Keeping them separate means checkout works for guests and the store never stores credentials itself.

Byte 4: Money in motion: CartItem, Order, OrderLine

Builds on: Bytes 1–3

In plain terms:

The cart is a scratchpad; the order is the receipt. Cart rows reference the live catalogue, while order lines snapshot names and prices at purchase time so later price edits cannot rewrite history.

The code:

```
csharp
public class CartItem {
    public string CartSessionId { get; set; }
    public int ProductId { get; set; }
    public int? ProductVariantId { get; set; }
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
}
public class Order {
    public string OrderNumber { get; set; }  // e.g. LE-20261011-482913
    public string IdempotencyKey { get; set; }
    public string Status { get; set; }
}
```

What's happening: CartItem is keyed by anonymous CartSessionId plus product/variant, with a frozen UnitPrice. Order snapshots the shipping address, SubTotal/ShippingCost/TaxAmount/GrandTotal, payment/shipping methods, Status (for example “Placed”), a unique IdempotencyKey against double submits, and Lines. OrderLine copies ProductName/VariantLabel/UnitPrice/LineTotal at purchase time.

Why it matters: Session carts need no login; idempotency keys make refresh/back-button safe; snapshotted lines keep admin revenue reports (SUM of LineTotal) honest. Gotcha: CartItem.UnitPrice is a starting point — checkout recomputes everything server-side before charging totals.

PUTTING IT TOGETHER

Categories tree into products; products fan out into images and variants; customers (optionally guests) fill session-keyed cart items; checkout freezes those lines into an order snapshot with an idempotency key. That single chain — catalogue → cart → order snapshot — is what the services, controllers and admin dashboard all operate on.

4 Contracts — Interfaces.cs and Common/Common.cs

These are C# interface and result-wrapper files: they declare what each layer can do without saying how. Assume the reader knows basic programming but is new to this codebase.

Byte 1: Generic persistence: IRepository<T> and IUnitOfWork

Builds on: None — starting point

In plain terms:

Think of the repository as a typed drawer for any entity (products, orders, …) and the unit of work as the "save everything together" button. Services program against these drawers, never against SQL.

The code:

```csharp
public interface IRepository<T> where T : class {
    IQueryable<T> Query();
    T GetById(object id);
    void Add(T entity);
    void Update(T entity);
    void Delete(T entity);
    int Count(Expression<Func<T, bool>> p);
}
```

What's happening: Query returns a composable LINQ query (filtering/sorting/paging happen database-side); GetById/Add/Update/Delete cover single-row work; Count takes a predicate. EfUnitOfWork hands out repositories sharing one DbContext and commits with SaveChanges.

Why it matters: Controllers and services stay testable and EF-agnostic. The IQueryable design is also why catalog paging stays fast: pages are sliced in SQL, not in memory.

Byte 2: Service boundaries: catalog, cart, checkout, account, admin

Builds on: Byte 1

In plain terms:

Each business capability gets one narrow interface, so a controller only sees the verbs it needs (search products, add to cart, place order) and cannot reach into the database directly.

The code:

```csharp
public interface ICatalogService {
    PagedResult<ProductCardViewModel> SearchProducts(string query, int? categoryId, string sort, int
page, int pageSize);
    ProductDetailViewModel GetProductDetail(string slugOrId, bool includeInactive = false);
}
public interface ICheckoutService {
    OperationResult<OrderConfirmationViewModel> PlaceOrder(string cartSessionId,
CheckoutFormViewModel form, string userEmail, string idempotencyKey);
}
```

What's happening: ICatalogService covers search/detail/tree/featured/new/discounted/category-page; ICartService covers get/add/update/remove/clear/count; ICheckoutService covers summary/place/get/history; IAccountService covers register/credential-check/customer-link; IAdminService

covers dashboard/stock/active/admin-check; IDatabaseHealthCheck exposes a single IsHealthy probe so controllers never touch EF.

Why it matters: Narrow interfaces enforce layering: business rules live in Services, HTTP lives in controllers. The includeInactive flag is the seam that lets admins preview hidden products while shoppers only see active ones.

Byte 3: Outcome wrappers: PagedResult<T> and OperationResult

Builds on: Byte 2

In plain terms:

Every list comes with its own page numbers, and every action reports success plus a human message. Views and AJAX handlers can rely on the same two shapes everywhere.

The code:

```
csharp
public int TotalPages {
    get { return PageSize <= 0 ? 0 : (TotalCount + PageSize - 1) / PageSize; }
}
public static OperationResult Fail(string message) { return new OperationResult { Success = false,
Message = message }; }
```

What's happening: PagedResult carries Items/TotalCount/Page/PageSize with a ceiling-division TotalPages (and a divide-by-zero guard). OperationResult carries Success/Message; the generic child adds Data. Static Ok/Fail factories keep call sites one-liners.

Why it matters: Pagination controls, “only N left” checks and toast messages all read these fields. Edge case: PageSize <= 0 yields 0 pages, which services clamp back to a default of 12 before querying.

PUTTING IT TOGETHER

Interfaces draw the map (repository → services → controllers) and the two wrapper classes standardise every answer: paged lists for browsing, success/message envelopes for actions. Read any service or controller through these shapes and its intent becomes obvious.

5 View models — ViewModels.cs and AdminViewModels.cs

These are C# data-transfer classes shaped for Razor views and JSON responses: flattened, display-ready copies of entities. Assume the reader knows basic programming but is new to MVC view models.

Byte 1: Shelf cards and detail pages

Builds on: None — starting point

In plain terms:

A product card is the tile you see in a grid (photo, price, discount, stock flag); the detail model is the full product page plus gallery, variant picker and related items.

The code:

```
csharp
public class ProductCardViewModel {
    public string ImageUrl { get; set; }
    public int? DiscountPct { get; set; }
    public bool InStock { get; set; }
}
public class ProductDetailViewModel {
    public IList<ProductImageVm> Images { get; set; }
    public IList<ProductVariantVm> Variants { get; set; }
    public IList<ProductCardViewModel> Related { get; set; }
}
```

What's happening: Cards precompute display fields (DiscountPct, InStock, CategoryName) so Razor does no arithmetic. The detail model adds Descriptions, stock/active flags, category slug for breadcrumbs, and three lists: images, variants, and up to four related cards from the same category.

Why it matters: Views stay logic-free and fast: one flat object per tile, one rich object per page. Related items reuse the card type, so the same partial renders everywhere.

Byte 2: Cart, checkout and order history shapes

Builds on: Byte 1

In plain terms:

The cart view shows each line with its photo and stock hint plus the four money rows (subtotal, shipping, tax, total); checkout pairs that cart with a validated address form; confirmations and history trim it back down to receipt rows.

The code:

```
csharp
public class CartLineViewModel {
    public decimal UnitPrice { get; set; }
    public int Quantity { get; set; }
    public decimal LineTotal { get; set; }
    public int StockAvailable { get; set; }
}
[Required, EmailAddress] public string Email { get; set; }  // on CheckoutFormViewModel
```

What's happening: CartLineViewModel carries names, slugs, image, variant label, prices and StockAvailable (used for “only N left”). CartViewModel aggregates SubTotal/Shipping/Tax/GrandTotal/ItemCount. CheckoutFormViewModel uses DataAnnotations ([Required], [EmailAddress], [StringLength]) so MVC

validates both server-side and, via unobtrusive jQuery, client-side. OrderConfirmation/OrderHistoryItem are receipt projections.

Why it matters: One cart model feeds the cart page, mini-cart, checkout summary and review step. Validation attributes are the single source of truth for address rules; gotcha: review/confirm reuses the posted form from session, so tampered totals in the browser can never change what the server charges.

Byte 3: Admin dashboard shapes

Builds on: Bytes 1–2

In plain terms:

The dashboard is a wall of counters plus three tables: recent orders, best sellers, and a 14-day sales strip. The view models carry exactly those numbers — plus the rulebook for how they are counted.

The code:

```csharp
public class AdminDashboardViewModel {
    public int LowStockThreshold { get; set; }
    public int LowStockProducts { get; set; }
    public decimal TotalRecordedSales { get; set; }
    public IList<BestSellerViewModel> BestSellers { get; set; }
}
```

What's happening: Counts cover products (total/active/inactive), categories, low/out-of-stock, orders (total/pending/completed), units sold and recorded sales (SUM of purchase-time LineTotal, never current prices). BestSellerViewModel and SalesByDayViewModel back the two breakdown tables; AdminStockUpdateViewModel validates the stock-edit form with [Range(0, 100000)].

Why it matters: Dashboard numbers are defined here and computed in AdminService: qualifying orders exclude Cancelled/Refunded, pending means Placed, low stock means 0 < stock < threshold. Since the store only creates Placed orders, completed counts read 0 until fulfilment statuses exist — by design, not a bug.

PUTTING IT TOGETHER

Cards for grids, detail for product pages, lines-plus-totals for every cart surface, validated forms for checkout input, receipts for history, and counters for admin. Each screen gets exactly the fields it renders — entities never leak into views, which is why price and stock rules can change in services without touching Razor.

6 Data layer — DbContext, repositories, health check

This is C# Entity Framework 6 code: the database session, the table mappings and a connectivity probe. Files: Ecommerce.Data/EcommerceDbContext.cs, Repositories/Repositories.cs, DatabaseHealthCheck.cs. (EcommerceModel.edmx is a retained design artefact; the running app uses the code-first mapping below so it works on Mono without EDMX codegen.)

Byte 1: One session for the whole shop: EcommerceDbContext

Builds on: None — starting point

In plain terms:

The context is the app's single doorway to SQL Server: one property per table, all sharing one connection and one change tracker.

The code:

```
csharp
public class EcommerceDbContext : DbContext {
    public EcommerceDbContext() : base("name=EcommerceDb") { }
    public virtual DbSet<Category> Categories { get; set; }
    public virtual DbSet<Order> Orders { get; set; }  // + Products, Images, Variants, Customers,
CartItems, OrderLines, Addresses
}
```

What's happening: Nine DbSet properties expose the nine tables; two constructors allow default (named connection string EcommerceDb) or explicit connection strings (used by tests/tooling). virtual supports test doubles.

Why it matters: Everything in one request shares one context instance (per-request lifetime, see Unity bytes), so a checkout that writes an order, decrements stock and clears the cart commits atomically via one SaveChanges.

Byte 2: Table rules in code: OnModelCreating

Builds on: Byte 1

In plain terms:

This method is the contract with the database: which table, which key, how long a name may be, how money precision is kept, and what happens to child rows when a parent is deleted.

The code:

```
csharp
modelBuilder.Entity<Product>().ToTable("Products").HasKey(p => p.ProductId);
modelBuilder.Entity<Product>().Property(p => p.Price).HasPrecision(18, 2);
modelBuilder.Entity<ProductImage>().HasRequired(i => i.Product)
    .WithMany(p => p.Images).HasForeignKey(i => i.ProductId).WillCascadeOnDelete(true);
```

What's happening: Categories get required names/slugs and a self-referencing optional parent; Products require name/slug, use DECIMAL(18,2) for all money, and must belong to a category; images/variants cascade-delete with their product; customers/orders/cart rows use optional links so guests survive; order lines cascade with their order.

Why it matters: It mirrors db/schema.sql 1:1. Money precision avoids rounding drift; cascade rules prevent orphan photos; optional customer links are what make guest checkout possible. Mismatch here vs. schema.sql would surface as runtime mapping errors.

Byte 3: Thin wrappers: EfRepository<T> and EfUnitOfWork

Builds on: Bytes 1–2

In plain terms:

These classes adapt the big DbContext to the small IRepository/IUnitOfWork interfaces: each method is a one-line delegation.

The code:

```
csharp
public IQueryable<T> Query() { return _set; }
public void Update(T entity) { _ctx.Entry(entity).State = EntityState.Modified; }
```

What's happening: EfRepository holds a DbSet<T> and forwards Query/GetById(Find)/Add/Update/Delete/Count. EfUnitOfWork creates repositories from the same context, forwards SaveChanges, and disposes the context.

Why it matters: Services stay interface-bound and mockable; disposal flows through the unit of work. Note the services in this codebase mostly use the DbContext directly and the repository pair stands ready for code that prefers the abstraction.

Byte 4: Safe health probe: DatabaseHealthCheck

Builds on: Byte 1

In plain terms:

A tiny class that answers one question — “can I reach the database right now?” — without leaking any details when the answer is no.

The code:

```
csharp
_db.Database.Connection.Open();
_db.Database.Connection.Close();
return true;
catch { return false; }
```

What's happening: Open/close the underlying connection inside try/catch; any failure returns false. It lives in the Data layer so controllers never touch EF (they depend on IDatabaseHealthCheck).

Why it matters: Powers GET /health for Docker/Nginx health checks: 200 OK versus 503 DB_UNAVAILABLE, with no credentials or exception text ever sent to the client.

PUTTING IT TOGETHER

One context maps nine tables with money-safe, guest-friendly rules; thin repository wrappers expose it through interfaces; a boolean probe reports liveness. The design artefact (.edmx) documents the model while the fluent mapping runs it — identical shapes, Mono-safe execution.

7 Services — business logic

This is C# service-layer code: the only place with business rules. Files: Mapping.cs, Impl/CatalogService.cs, Impl/CartService.cs, Impl/CheckoutAccountServices.cs, Impl/AdminService.cs. Assume the reader knows basic programming but is new to this store.

Byte 1: Shared display math: Mapping.ToCard and PricingRules

Builds on: None — starting point

In plain terms:

Two helpers everyone reuses: turn a product row into a shelf card (with discount badge and first photo), and compute shipping, tax and variant prices from one central place.

The code:

```
csharp
if (p.CompareAtPrice.HasValue && p.CompareAtPrice.Value > p.Price && p.CompareAtPrice.Value > 0)
    discount = (int)Math.Round((double)((p.CompareAtPrice.Value - p.Price) / p.CompareAtPrice.Value
* 100));
public static decimal ShippingFor(decimal subtotal, string method) {
    if (subtotal <= 0) return 0;
    if (m == "express") return subtotal >= 150 ? 0 : 12.99m;
    return subtotal >= 75 ? 0 : 5.99m; // standard
}
```

What's happening: ToCard prefers the CompareAtPrice-derived percentage, falls back to DiscountPercent, picks the first image (or a placeholder), and flags InStock from StockQuantity. PricingRules encodes the demo tariff: standard free over ₹75 else ₹5.99, express ₹12.99/free over ₹150, priority ₹24.99/free over ₹200, 8% tax, and EffectivePrice = base + variant adjustment. Server-side only.

Why it matters: One definition of “price” everywhere — cards, cart, checkout and admin agree. Empty carts ship for ₹0 by rule; negative/zero compare prices never produce a discount.

Byte 2: Browsing: CatalogService search, sort and paging

Builds on: Byte 1

In plain terms:

Search is “filter, then sort, then slice one page”. The service clamps bad page numbers, widens a category to include its children, and always applies a deterministic tie-break so pages never shuffle.

The code:

```
csharp
if (pageSize < 1 || pageSize > 60) pageSize = 12;
q = q.Where(p => p.Name.ToLower().Contains(s) || p.Description.ToLower().Contains(s) ||
p.Sku.ToLower().Contains(s));
case "price_asc": return q.OrderBy(p => p.Price).ThenBy(p => p.ProductId);
```

What's happening: Only IsActive products are queried with Category/Images eagerly loaded. Category filter adds direct children one level deep. Sort keys: price/name asc/desc, newest, and default recommended (featured → new → newest, always ending in ProductId). Over-range pages clamp to the last page. GetCategoryPage repeats the pattern scoped to a slug’s subtree; GetCategoryTree builds roots/children in memory and rolls child counts into parents; featured/new/discounted are small curated takes.

Why it matters: Stable secondary keys keep pagination and infinite scroll duplicate-free. Case-insensitive Contains across name/description/SKU is simple but not full-text — fine for a demo catalog, a known scaling limit.

Byte 3: Product pages: GetProductDetail

Builds on: Byte 2

In plain terms:

One method serves both shoppers and admins: look a product up by slug or numeric id, attach its gallery, variants and related items, and guarantee there is always at least a placeholder photo.

The code:

```
csharp
if (int.TryParse(slugOrId, out id))
    p = _db.Products.Include(x => x.Category).Include(x => x.Images).Include(x => x.Variants)
        .FirstOrDefault(x => x.ProductId == id && (includeInactive || x.IsActive));
if (vm.Images.Count == 0) vm.Images.Add(new ProductImageVm { ImageUrl = "/Images/placeholder.png",
AltText = p.Name });
```

What's happening: Slug and id paths are both supported; includeInactive (default false) lets the admin inspector view hidden products. Discount is recomputed, images ordered by DisplayOrder, variants projected with per-variant stock, and up to four same-category siblings fill Related.

Why it matters: Controllers stay trivial (fetch-or-404). The dual lookup is why /p/42 and /p/classic-cotton-kurta both work, and why deactivated products vanish from the store but remain inspectable.

Byte 4: Carts: CartService with stock-aware guards

Builds on: Bytes 1–3

In plain terms:

Adding to cart is not just an insert — the service checks the product is live, the variant exists, stock covers the merged quantity, and per-add caps hold. Every mutation re-saves and every read recomputes totals.

The code:

```
csharp
if (quantity > 99) return OperationResult.Fail("Maximum 99 units per add.");
int newQty = quantity + (existing != null ? existing.Quantity : 0);
if (newQty > available) return OperationResult.Fail("Only " + available + " unit(s) available.");
```

What's happening: GetCart joins items to products/variants, resolves images and variant labels, and derives SubTotal/Shipping/Tax/GrandTotal/ItemCount. AddToCart merges with an existing line for the same session+product+variant and refreshes UnitPrice to the effective price. UpdateQuantity treats 0 as remove; Remove/Clear delete; GetCartCount sums quantities.

Why it matters: Oversell is blocked at the service edge (merged quantities included), and unit prices always refresh to current effective prices. Gotcha: stock is checked, not reserved — two shoppers can race the last unit; checkout re-validates before committing.

Byte 5: Checkout: idempotent, server-recomputed PlaceOrder

Builds on: Byte 4

In plain terms:

Checkout never trusts browser totals: it reloads every product, rechecks stock, recomputes money, then writes the order, decrements stock and empties the cart. A client-supplied idempotency key makes repeats return the original order instead of a duplicate.

The code:

```csharp
var dup = _db.Orders.Include(o => o.Lines).FirstOrDefault(o => o.IdempotencyKey == idempotencyKey);
if (dup != null) return OperationResult<OrderConfirmationViewModel>.Ok(ToConfirmation(dup),
"Duplicate submission — returning original order.");
decimal unit = PricingRules.EffectivePrice(product, variant);
```

What's happening: BuildSummary pre-fills country/shipping/payment defaults. PlaceOrder generates a key when missing, short-circuits duplicates, fails empty carts, validates each line against live stock, whitelists shipping/payment methods (standard/express/priority; cod/card_demo/upi_demo/netbanking_demo), links the customer by email, numbers the order LE-yyyyMMdd-xxxxxx, decrements variant and product stock (floored at 0), saves once, then clears the cart. GetOrder/GetOrderHistory project confirmations and receipt rows.

Why it matters: Double-clicks, refreshes and back-button resubmits cannot create duplicate charges-or-records. Random order numbers can theoretically collide (no retry loop) and stock decrement is not transactionally isolated — acceptable demo trade-offs, worth knowing before scaling.

Byte 6: Accounts and admin: thin but strict

Builds on: Bytes 1, 5

In plain terms:

Passwords belong to ASP.NET Identity; this service only keeps the shop-side customer row in sync. The admin service counts money from frozen order lines and guards its two write actions with ranges.

The code:

```csharp
public void EnsureCustomerRecord(string aspNetUserId, string email, string firstName, string
lastName) {
    var c = _db.Customers.FirstOrDefault(x => x.Email == email);
    if (c == null) { _db.Customers.Add(new Customer { ... }); _db.SaveChanges(); }
}
public bool IsAdmin(string email) {
    const string sql = @"SELECT CASE WHEN EXISTS( SELECT 1 FROM dbo.AspNetUsers u ... WHERE u.Email
= @p0 AND r.Name = 'Admin') ...";
    return _db.Database.SqlQuery<int>(sql, email).FirstOrDefault() == 1;
}
```

What's happening: AccountService.Register delegates credential creation to Identity and ensures the Customer row; ValidateCredentials is a deliberate stub (Identity handles real auth). AdminService.GetDashboard aggregates counts and SUMs database-side, excluding Cancelled/Refunded lines and grouping best sellers and 14-day sales; UpdateStock enforces 0–100000; SetActive toggles visibility; IsAdmin checks the Admin role with a parameterised query (@p0, no string concatenation).

Why it matters: No password logic to get wrong in shop code; no SQL-injection surface in the role check; dashboard revenue always reflects purchase-time prices. Admin writes are deliberately minimal: stock and visibility only — no price editing from the UI.

PUTTING IT TOGETHER

Mapping and PricingRules give every layer the same numbers; CatalogService makes browsing stable and paged; CartService guards quantities against live stock; CheckoutService re-validates and de-duplicates order placement; Account/Admin services link identity to customers and report on frozen history. Controllers therefore contain almost no logic — they translate HTTP into these calls and render what comes back.

8 Web composition — startup, DI, auth, session

This is C# ASP.NET MVC 5 / OWIN plumbing: how requests find controllers, where services come from, who the user is, and which cart belongs to whom. Files: Global.asax(.cs), Startup.cs, App_Start/RouteConfig.cs, UnityConfig.cs, BundleConfig.cs, FilterConfig.cs, Infrastructure/UnityResolver.cs, CartSession.cs, AdminBootstrap.cs, Models/IdentityModels.cs.

Byte 1: Application birth and death: Global.asax.cs

Builds on: None — starting point

In plain terms:

When the app starts it registers areas, filters, routes, bundles and the DI container, then optionally provisions an admin; when each request ends it cleans up that request's DI scope.

The code:

```
csharp
UnityConfig.RegisterComponents();
Infrastructure.AdminBootstrap.Run();
protected void Application_EndRequest() {
    Infrastructure.UnityResolver.DisposeChildContainer();
}
```

What's happening: Application_Start wires the whole pipeline in a fixed order; Application_Error traces (never displays) unhandled exceptions while customErrors shows a friendly page; Application_EndRequest disposes the per-request Unity child container null-safely.

Why it matters: Startup ordering bugs (for example routes before DI) are a classic MVC failure — this file makes the order explicit. The null-safe teardown is what lets the app survive Mono/XSP quirks without leaking containers.

Byte 2: URLs: RouteConfig's four rules

Builds on: Byte 1

In plain terms:

Four routes decide every URL: health probes, pretty product/category addresses, and a conventional fallback for everything else.

The code:

```
csharp
routes.MapRoute("Health", "health", new { controller = "Home", action = "Health" });
routes.MapRoute("Product", "p/{slug}", new { controller = "Catalog", action = "Product" });
routes.MapRoute("Category", "c/{slug}", new { controller = "Catalog", action = "Category" });
```

What's happening: IgnoreRoute skips .axd handlers; /health hits Home.Health; /p/{slug} and /c/{slug} give short memorable product/category URLs; {controller}/{action}/{id} with Home/Index defaults catches the rest. Registration order matters — specific routes precede the default.

Why it matters: Pretty URLs are why views link to /p/classic-cotton-kurta instead of /Catalog/Product?slug=…. Adding a new pretty route means inserting it before Default, or the generic rule will swallow it.

Byte 3: Wiring: UnityConfig and the Mono-safe UnityResolver

Builds on: Bytes 1–2

In plain terms:

Controllers declare what they need (for example ICatalogService); Unity builds it per request with a fresh DbContext. A custom resolver replaces the stock per-request module that crashes under Mono/XSP.

The code:

```
csharp
container.RegisterType<EcommerceDbContext>(new HierarchicalLifetimeManager());
container.RegisterType<ICatalogService, CatalogService>(new HierarchicalLifetimeManager());
DependencyResolver.SetResolver(new UnityResolver(container));
```

What's happening: All contexts and services register with HierarchicalLifetimeManager (one instance per child container). UnityResolver keeps a child container in HttpContext.Items, resolves through it, returns null instead of throwing for unknown types, and DisposeChildContainer tears it down defensively.

Why it matters: Per-request scoping is what makes one DbContext == one atomic SaveChanges per request possible. The custom resolver exists for a verified Mono/XSP NullReference on teardown — do not “upgrade” it back to the stock module without retesting on Mono.

Byte 4: Identity: users, passwords and cookies

Builds on: Byte 3

In plain terms:

Login state comes from OWIN cookie authentication layered over ASP.NET Identity: user rows, hashed passwords, lockout counters and roles live in the AspNet* tables.

The code:

```
csharp
manager.PasswordValidator = new PasswordValidator { RequiredLength = 6, ... };
manager.MaxFailedAccessAttemptsBeforeLockout = 5;
app.UseCookieAuthentication(new CookieAuthenticationOptions {
    LoginPath = new PathString("/Account/Login"), ExpireTimeSpan = System.TimeSpan.FromDays(7), ...
});
```

What's happening: ApplicationUser extends IdentityUser with first/last names; ApplicationDbContext reuses the EcommerceDb connection string; UserManager requires unique email, 6+ character passwords and 5-strikes/5-minute lockout; the cookie lasts 7 days with sliding renewal, HttpOnly, and a login redirect. RoleManager is registered so role claims (for example the Admin nav link) reach the cookie.

Why it matters: Auth display (header links) reads the cookie; authorisation for /Admin re-checks the database per request (see AdminController) so role revocation takes effect immediately. Passwords are hashed by Identity — the shop never sees them.

Byte 5: Anonymity plus safety: CartSession and idempotency keys

Builds on: Bytes 1–4

In plain terms:

Guests get a random cart id stored in server session; checkout gets a second random key that travels with the form and makes double submits harmless.

The code:

```csharp
private const string Key = "LE_CartId";
if (session[Key] == null) session[Key] = System.Guid.NewGuid().ToString("N");
return (string)session[Key];
```

What's happening: GetCartId lazily mints a GUID-hex cart id per session (two overloads cover both session abstractions); NewIdempotencyKey mints and stores a fresh checkout key; Peek reads it without rotating. Controllers set Session["LE_CheckoutKey"] on the checkout form and rotate it after a successful Confirm.

Why it matters: No login is needed to shop, yet refresh/back/double-click cannot duplicate orders. Session is InProc with a 60-minute timeout (see Web.config): cart ids vanish on app recycle — fine for a demo, a Redis/SQL session store would be the scale-up fix.

Byte 6: Opt-in power: AdminBootstrap

Builds on: Byte 4

In plain terms:

No admin exists unless the operator explicitly provides both an email and a password to the web process. Otherwise the Admin role is never used and /Admin stays unreachable.

The code:

```csharp
var email = Environment.GetEnvironmentVariable("ADMIN_EMAIL");
if (string.IsNullOrWhiteSpace(email)) { Status = "skipped: ADMIN_EMAIL not set"; return; }
if (!roleManager.RoleExists(RoleName)) roleManager.Create(new IdentityRole(RoleName));
```

What's happening: Run reads ADMIN_EMAIL/ADMIN_PASSWORD, creates the Admin role and user (same 6-character minimum policy) with EmailConfirmed, assigns the role idempotently, records a Status string, and never throws — any exception becomes a status message so startup always continues.

Why it matters: Public deployments simply leave both variables blank and gain no default credential to attack. Compose passes them through as optional (possibly empty) variables; only someone with environment access can bootstrap an admin.

PUTTING IT TOGETHER

Startup order (filters → routes → bundles → DI → admin bootstrap), four URL rules, per-request DI with Mono-safe teardown, cookie identity with DB-backed roles, session cart plus idempotency keys, and an admin that only exists when explicitly requested. Together they form the request pipeline every controller below plugs into.

9 Controllers — HTTP in, views out

This is C# ASP.NET MVC 5 controller code: thin adapters that call services and pick views. Six files under Ecommerce.Web/Controllers/. Assume the reader knows basic programming but is new to MVC.

Byte 1: HomeController: storefront landing plus health probe

Builds on: None — starting point

In plain terms:

The home page is three curated product lists plus the category tree; /health is a plain-text load-balancer probe that also checks the database.

The code:

```
csharp
ViewBag.Featured = _catalog.GetFeatured(8);
ViewBag.NewArrivals = _catalog.GetNewArrivals(8);
ViewBag.Discounted = _catalog.GetDiscounted(4);
if (_health.IsHealthy()) return Content("OK", "text/plain");
Response.StatusCode = 503; return Content("DB_UNAVAILABLE", "text/plain");
```

What's happening: Index fills ViewBags and returns the view; Health returns OK or a 503 with a fixed string — never exception text. About/Contact/Shipping/Returns/Privacy/Error serve static pages.

Why it matters: Docker and Nginx health checks depend on this exact contract (same URL, body and codes). Keeping DB errors out of the response avoids leaking infrastructure details.

Byte 2: CatalogController: search, category, product and AJAX grids

Builds on: Byte 1 (plus section 7, byte 2)

In plain terms:

One controller serves full pages, filtered-grid partials for AJAX refresh, and JSON plus rendered HTML for infinite scroll. Page size is whitelisted to 12 or 24.

The code:

```
csharp
private static int PageSize(int? size) {
    if (!size.HasValue) return 12;
    if (size.Value == 12 || size.Value == 24) return size.Value;
    return 12;
}
return Json(new { html = html, hasMore = hasMore, nextPage = actualPage + 1, total = total },
JsonRequestBehavior.AllowGet);
```

What's happening: Index/Category/Product resolve ViewBags (query, sort, size, mode) and return 404 for unknown slugs. Children returns lazy nav children; Grid returns the _ProductGrid partial; GridData server-renders the same partial to a string and pairs it with hasMore/nextPage/total (never more than one page — service limits apply). RenderPartialToString uses the view engine directly and returns empty on failure.

Why it matters: Full reload, AJAX filter and infinite scroll all share one service path, so results agree. AllowGet on JSON is intentional here (read-only paging data); anti-forgery applies to the POST actions, not these GETs.

Byte 3: CartController: AJAX-or-redirect mutations

Builds on: Section 8, byte 5 (CartSession)

In plain terms:

Add, update and remove accept normal posts and AJAX: AJAX callers get JSON or a refreshed cart partial; plain forms get a redirect with a one-time message.

The code:

```csharp
var r = _carts.AddToCart(CartId, productId, variantId, quantity);
if (Request.IsAjaxRequest())
    return Json(new { success = r.Success, message = r.Message, count = _carts.GetCartCount(CartId)
});
TempData[r.Success ? "Notice" : "Error"] = r.Message;
```

What's happening: Every POST carries [ValidateAntiForgeryToken]. CartId comes from CartSession. Update/Remove return the _CartLines partial with the message tucked into ViewBag for toast display. Mini/Count feed the header mini-cart and badge.

Why it matters: Progressive enhancement: the cart works with JavaScript disabled (redirects) and feels instant with it (partial swaps). TempData-versus-partial is the fork that makes both paths share one service result.

Byte 4: CheckoutController: a five-step flow that only writes once

Builds on: Bytes 2–3 (plus service byte 5)

In plain terms:

Details form → validate → park the form in session → review totals → confirm and place. Only Confirm calls PlaceOrder; viewing Review never creates anything.

The code:

```csharp
Session["LE_PendingForm"] = form;
return RedirectToAction("Review");
// Confirm:
var result = _checkout.PlaceOrder(CartId, form, email, idempotencyKey);
Session.Remove("LE_PendingForm");
Session["LE_CheckoutKey"] = CartSession.NewIdempotencyKey(Session);
```

What's happening: Index redirects empty carts away, prefills the email for logged-in users and issues the idempotency key. Place validates; on failure it rebuilds the summary with the posted form, on success it stores the form and redirects (Post/Redirect/Get). Review re-renders server totals read-only. Confirm consumes the session form exactly once, rotates the key, and redirects to Success; Success reloads the order by number or 404s.

Why it matters: GETs never mutate; the key rotation plus PlaceOrder's duplicate guard gives two independent layers against double orders. Losing session state mid-flow (recycle/timeout) safely restarts checkout instead of half-placing an order.

Byte 5: AccountController: Identity behind a Mono-safe sync bridge

Builds on: Section 8, byte 4

In plain terms:

Register, login with lockout handling, profile/order history, and logoff. Identity's async methods are called synchronously through a pool-thread bridge because HttpContext does not flow across awaits on Mono/XSP.

The code:

```
csharp
private static T RunSync<T>(System.Func<Task<T>> fn) { return Task.Run(fn).GetAwaiter().GetResult();
}
var user = RunSync(() => manager.FindByEmailAsync(model.Email));
if (RunSync(() => manager.IsLockedOutAsync(user.Id))) { ModelState.AddModelError("", "Account
locked. Try again later."); ... }
_accounts.EnsureCustomerRecord(user.Id, user.Email, null, null);
```

What's happening: Login checks lockout, verifies the password, resets the failure count, builds the identity and signs in with optional persistence; failures increment the counter and show a generic message. Register creates the Identity user, links the Customer row, and signs in immediately. Profile/Orders read history by the logged-in email; RedirectToLocal only follows local URLs (open-redirect guard).

Why it matters: The RunSync bridge avoids deadlocks without touching business logic — cookie sign-in still runs on the request thread. Generic “invalid email or password” messages avoid disclosing which accounts exist.

Byte 6: AdminController: role gate on every action

Builds on: Section 8, bytes 4 and 6

In plain terms:

Every admin page first requires login, then re-checks the Admin role in the database on each request. Non-admins get 403; anonymous users are bounced to login.

The code:

```
csharp
protected override void OnActionExecuting(ActionExecutingContext filterContext) {
    base.OnActionExecuting(filterContext);
    if (User.Identity.IsAuthenticated && !_admin.IsAdmin(User.Identity.Name))
        filterContext.Result = new HttpStatusCodeResult(403, "Administrators only.");
}
public ActionResult Index(int threshold = 10) { ... return View(_admin.GetDashboard(threshold)); }
```

What's happening: [Authorize] handles the anonymous case; OnActionExecuting adds the per-request role check. Index renders the dashboard; Products reuses catalog search for paged admin filtering; Product shows the inspector (including inactive items); UpdateStock validates the range then applies stock and active flag together with combined feedback.

Why it matters: Authorisation never relies on the cookie’s role claim alone, so demoting a user bites immediately. Admin reads reuse the catalog service instead of duplicating query logic — one search implementation serves shoppers and staff.

PUTTING IT TOGETHER

Home curates, Catalog browses (pages, partials and JSON from one source), Cart mutates with dual AJAX/redirect responses, Checkout stages input in session and writes exactly once, Account bridges Identity safely, and Admin re-verifies roles per request. Every controller is a thin HTTP shell over the service bytes from section 7.

10 Views — Razor pages and partials

These are Razor (.cshtml) templates: HTML mixed with C# that renders the storefront. About 30 files under Ecommerce.Web/Views/. This section groups them so each byte stays readable; every file covered is named. Assume the reader knows basic HTML but is new to Razor.

Byte 1: Shell: _Layout, _ViewStart, _Header, _Footer

Builds on: None — starting point

In plain terms:

One layout wraps every page: icon rail, header, content, footer, then scripts. A single anti-forgery token is minted per request and shared by every form on the page.

The code:

```razor
Layout = "~/Views/Shared/_Layout.cshtml";
Context.Items["AF_TOKEN_HTML"] = Html.AntiForgeryToken().ToHtmlString();
@Html.Partial("_Header")
@RenderBody()
@RenderSection("scripts", required: false)
```

What's happening: _ViewStart sets the layout and generates the one token (verified on Mono/XSP: only the first token per response validates, so AJAX-loaded cards stamp the page token before submit — see site.js). _Layout renders TempData Notice/Error banners, the slim icon rail with active states, and script bundles. _Header holds brand, live title, search box, auth-aware links (including the Admin link only for role members), the cart button with count badge, the mini-cart panel shell, and the category strip with hover dropdowns. _Footer groups shop/account/help/company links and the INR fine print. (Files: _Layout.cshtml, _ViewStart.cshtml, _Header.cshtml, _Footer.cshtml, Views/Web.config registers the Razor host.)

Why it matters: A single token kills the classic “multiple tokens, random 400s” Mono bug. Auth-aware header links are affordances only — real gating stays server-side in AdminController.

Byte 2: Reusable product chrome: _ProductCard, _ProductGrid, _CategoryChildren

Builds on: Byte 1

In plain terms:

The card is the atom of the whole store: photo with discount/out-of-stock badge, category eyebrow, two-line name, price row, and an add-to-cart form (or a disabled button when unavailable). The grid just repeats it; the children partial fills lazy nav dropdowns.

The code:

```razor
<article class="yb-card" data-pid="@Model.ProductId">
    <a class="yb-card-img" href="/p/@Model.Slug"><img ...
onerror="this.src='/Images/placeholder.png'" /></a>
    <button type="submit" ... data-name="@Model.Name">Add to cart</button>
</article>
```

What's happening: Cards link by slug, show CompareAtPrice strikethrough and “N% off”, carry data-pid for infinite-scroll dedupe, and post productId/quantity with the shared token. _ProductGrid renders the empty state (“No products found” + clear-filters) or the .yb-grid of cards. _CategoryChildren renders lazy child links for hover-loaded menus. (Files: _ProductCard.cshtml, _ProductGrid.cshtml, _CategoryChildren.cshtml.)

Why it matters: One card partial guarantees home, search, category, related-items and AJAX pages look and behave identically. Image onerror fallbacks mean a missing photo degrades to a placeholder instead of a broken tile.

Byte 3: Discovery pages: Home/Index, Catalog/Index, Catalog/Category, Catalog/Product

Builds on: Bytes 1–2

In plain terms:

Home shows category tiles plus featured/new/discounted rails; the two browse pages share a filter bar (sort, page size, pagination/infinite toggle) with a sidebar of departments; the product page pairs a gallery with variant-aware buying.

The code:

```razor
<script>YB_GRID = { slug: '@Model.Slug', sort: '@sort', size: @size, mode: '@mode', ... };</script>
<select name="sort" id="sort-sel"> ... <option value="price_asc">Price low-high</option> ...
<div id="infinite-zone" data-next="@(Model.Page + 1)" data-hasmore="...">
```

What's happening: Category icons resolve from slug keywords (footwear/fashion/decor/home/tech/beauty/sports/books/toys) with a placeholder fallback. Browse pages cap pager links at 20, expose YB_GRID for site.js, and render either a pager or the infinite-zone with Load more. Product.cshtml swaps the main image on thumbnail click, syncs variant price/stock client-side (server re-validates on add), and reuses _ProductGrid for related items. (Files: Home/Index.cshtml, Catalog/Index.cshtml, Catalog/Category.cshtml, Catalog/Product.cshtml.)

Why it matters: Server renders first paint; JavaScript progressively enhances (AJAX sort/pager, deduped infinite scroll). Variant stock disables the buy button live, but the authoritative check stays in CartService.

Byte 4: Cart surfaces: Cart/Index, _CartLines, _MiniCart

Builds on: Bytes 2–3 (plus controller byte 3)

In plain terms:

The full cart page is a one-line wrapper around the _CartLines partial — the same partial AJAX updates swap in after quantity edits. The mini-cart shows the first four lines anywhere on the site.

The code:

```razor
<div id="cart-lines" data-message="@msg" data-ok="...">
    <input type="number" class="form-control yb-qty" value="@l.Quantity" min="0" max="99" ... />
    <div class="yb-totals" aria-live="polite"> ... Tax (8%) ... </div>
```

What's happening: _CartLines renders the empty state or the item table (thumbnail, name, variant, low-stock hint at ≤5 units), quantity steppers, per-line totals, and the subtotal/shipping/tax/grand-total box with a checkout button (suppressible via ViewBag.HideCartActions for the review step). data-message/data-ok ferry the result message to toast code. _MiniCart caps at four rows with a “+N more” note. (Files: Cart/Index.cshtml, Shared/_CartLines.cshtml, Shared/_MiniCart.cshtml.)

Why it matters: Rendering the same partial for full page and AJAX keeps totals consistent everywhere. aria-live on totals and labelled quantity groups keep the cart usable with assistive tech.

Byte 5: Money steps: Checkout/Index, Review, Success

Builds on: Byte 4 (plus controller byte 4)

In plain terms:

Three pages walk the five labelled steps: address plus shipping/payment choice, then a read-only review of server-calculated totals, then a receipt. Demo payment labels are explicit that no real charge happens.

The code:

```
razor
<input type="hidden" name="idempotencyKey" value="@ViewBag.IdempotencyKey" />
<input type="radio" name="Form.ShippingMethod" value="express" /> ... ₹12.99 (free over ₹150)
<form method="post" action="/Checkout/Confirm" id="confirm-form">
```

What's happening: Index posts to /Checkout/Place with validation messages per field; Review displays the session-held form beside the totals (cart actions hidden) and posts to Confirm; Success shows order number, total, status and line table with continue/history links. Step pills (Details → Review → Done) orient the shopper. (Files: Checkout/Index.cshtml, Review.cshtml, Success.cshtml.)

Why it matters: The hidden idempotency key travels with the flow and is rotated after Confirm; shipping radio values mirror PricingRules tiers so displayed and charged amounts agree.

Byte 6: Accounts and admin screens

Builds on: Bytes 1, 5 (plus controller bytes 5–6)

In plain terms:

Auth pages pair a marketing side panel with a validated form; profile/orders show receipts; admin pages show dashboard counters with their rulebook, a filterable product table, and a stock-plus-visibility editor.

The code:

```
razor
@using (Html.BeginForm("Login", "Account", new { ReturnUrl = ViewBag.ReturnUrl }, FormMethod.Post))
{
@Html.ValidationMessageFor(m => m.Email, "", new { @class = "text-danger" })
<div class="yb-stats"> ... Low stock (&lt; @Model.LowStockThreshold) ... </div>
```

What's happening: Login/Register use Html helpers with per-field messages and alt links; Profile greets by email with recent orders; Orders links each number to its Success receipt. Admin/Index prints stat tiles, the metric-rules box, best sellers, recent orders and the 14-day table; Products.cshtml filters by text/category/sort with Inspect links; Product.cshtml shows gallery, variant stocks and the stock/active form. (Files: Account/Login, Register, Profile, Orders; Admin/Index, Products, Product; Home/About, Contact, Shipping, Returns, Privacy; Shared/Error.)

Why it matters: Admin metric rules are printed on the page itself, so reported numbers are self-documenting. The product editor's checkbox posts IsActive alongside stock — uncheck plus Save hides a product without deleting it.

PUTTING IT TOGETHER

One layout and one token serve every page; cards and grids repeat the same atom from home to AJAX payloads; browse pages degrade gracefully between pager and infinite scroll; cart, checkout and admin screens all render service-computed models with no arithmetic in Razor. Learn the card, the cart-lines partial and the checkout trio and you can navigate the entire storefront.

11 Client behaviour and styling — site.js and site.css

This is JavaScript (jQuery 3.4.1, no build step) plus a design-system stylesheet. Files: Ecommerce.Web/Scripts/site.js, Ecommerce.Web/Content/site.css. (Bootstrap/jQuery vendor copies are used, not documented.)

Byte 1: Resilient cart AJAX: tokens, toasts, counts

Builds on: None — starting point

In plain terms:

Every add/update/remove goes through one AJAX path that stamps the shared page token, disables the button while busy, and reports back with toasts plus refreshed badges — with plain-English errors when the network fails.

The code:

```
javascript
function stampToken($f) {
    var t = token();
    if (t) $f.find('input[name="__RequestVerificationToken"]').val(t);
    return t;
}
$.ajax({ url: $f.attr('action') || '/Cart/Add', method: 'POST', data: $f.serialize() })
```

What's happening: token reads the first page token; stampToken copies it into any submitting form (including AJAX-loaded cards whose own field renders empty). notify/showError render dismissible toasts and mirror errors into any alert bar. refreshCount/refreshMini keep the header badge and dropdown current; setBusy/cartMessage prevent double clicks and surface partial-render messages.

Why it matters: This is the client half of the single-token contract — without stamping, Mono/XSP would reject AJAX adds with 400s. Busy flags plus server idempotency give two layers of double-submit protection.

Byte 2: Grids without reloads: filters, pager, infinite scroll

Builds on: Byte 1

In plain terms:

Browse pages enhance in place: sort/size changes and pager clicks fetch just the grid partial; infinite mode appends server-rendered cards page by page while skipping duplicates already on screen.

The code:

```
javascript
var inf = { busy: false, seen: {} };
function gridDataUrl(page) { var g = gridParams();
    var url = '/Catalog/GridData?page=' + page + '&sort=' + ...;
    if (g.slug) url += '&slug=' + encodeURIComponent(g.slug); ...
    return url; }
```

What's happening: YB_GRID (rendered per page) supplies slug/query/category/sort/size/mode. markSeen tracks data-pid values; loadMore single-flights requests, strips already-seen cards, appends the rest, updates hasMore/nextPage, and offers Retry on failure. Scroll auto-load is debounced and stops when hasMore flips false; sort/pager handlers no-op in the opposite mode so full navigation still works.

Why it matters: Dedup by product id keeps interleaved first pages photo-diverse even if the server overlaps. Single-flight plus busy flags prevent request storms from fast scrolling.

Byte 3: Checkout, variants, gallery and menus

Builds on: Bytes 1–2

In plain terms:

Small focused handlers: lock the checkout button after a valid submit, recompute variant price/stock live, swap gallery images, and lazily load submenu children on first hover.

The code:

```javascript
var adj = parseFloat($opt.attr('data-adj')) || 0;
$('#buy-price').text('₹' + (base + adj).toFixed(2));
$('#buy-form [type="submit"]').prop('disabled', stock <= 0);
$(document).on('submit', '#checkout-form, #confirm-form', function () { ... })
```

What's happening: syncVariant reads data-base-price/data-adj/data-stock from the selected option; thumbnails swap #main-img; quantity steppers clamp 0–99 and debounce the AJAX update; Escape closes the mini-cart; category hover fetches /Catalog/Children once per menu.

Why it matters: Client variant math is display-only — CartService re-validates on add. Lazy submenu loading keeps the first paint light while deep nav stays one hover away.

Byte 4: The stylesheet contract: tokens plus yb-* hooks

Builds on: Bytes 1–3

In plain terms:

One file defines the whole look: CSS variables for colour and shadow, then components (tiles, cards, filter bar, cart table, mini-cart, forms, toasts, admin stats, footer). Every interactive class is referenced by Razor and/or site.js — renaming breaks behaviour, not just looks.

The code:

```css
:root {
    --yb-accent: #159d8c;
    --yb-accent-d: #0e7c6f;
    --yb-ink: #252b32;
    --yb-line: #e8ecef;
}
.yb-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(210px, 1fr)); gap: 14px; }
```

What's happening: Responsive grids collapse 4 → 3 → 2 columns; the pager hides middle links on mobile; focus outlines, skip links, aria hooks and reduced-motion support are built in. Bootstrap 3.4.1 supplies the base; site.css layers the mint/teal brand, sticky totals on desktop, and fixed mini-cart on small screens.

Why it matters: Design tokens keep branding in one place; the documented “do not rename” rule on .yb-* selectors is load-bearing because JavaScript queries those exact hooks. Gotcha for maintainers: visual edits are safe, selector renames require paired Razor/JS edits.

PUTTING IT TOGETHER

site.js makes the server-rendered pages feel instant (token-stamped cart AJAX, partial grid refreshes, deduped infinite scroll, live variant pricing) while site.css gives every state a consistent, responsive, accessible presentation. Both layers assume the server remains authoritative — the client enhances, never decides, prices or stock.

12 Database and seed data — db/

This is SQL Server DDL plus Python seed generators and shell orchestration. Files: schema.sql, init.sh, gen_seed.py, gen_seed1500.py, seed.sql, seed_1000.sql, seed_1500.sql, seed_product_images.sql. (Generated seed data files are large — tens of thousands of lines — and are characterised here rather than quoted in full.)

Byte 1: Idempotent schema: create-if-missing tables and indexes

Builds on: None — starting point

In plain terms:

The schema script can run any number of times safely: it creates the database and each table only when absent, and never drops anything.

The code:

```sql
IF DB_ID('LegacyEcommerceDb') IS NULL
BEGIN
    CREATE DATABASE LegacyEcommerceDb;
END
GO
IF OBJECT_ID('dbo.Products','U') IS NULL
CREATE TABLE dbo.Products(
    Price DECIMAL(18,2) NOT NULL,
    Slug NVARCHAR(250) NOT NULL UNIQUE, ...
);
```

What's happening: Nine shop tables (Categories with self-referencing parent; Products with SKU/slug/prices/stock/flags; ProductImages and ProductVariants with cascading deletes; Customers with unique email; CartItems; Orders with unique OrderNumber and unique IdempotencyKey; OrderLines cascading; Addresses) plus the five ASP.NET Identity tables and lookup indexes (category, slug, session, email).

Why it matters: Re-runnable deploys and container restarts cannot wipe data. UNIQUE on Slug/OrderNumber/IdempotencyKey enforces at the database what the services assume: addressable products and duplicate-proof orders.

Byte 2: One-shot init: init.sh waits, then applies schema plus seed

Builds on: Byte 1

In plain terms:

The dbinit container polls SQL Server until it answers, then applies the schema and exactly one catalog seed, followed by the product-photo fixup.

The code:

```shell
for i in $(seq 1 60); do
  if $SQLCMD -Q "SELECT 1" -b > /dev/null 2>&1; then
    echo "SQL Server is up."; break;
  fi
  ... sleep 5
done
```

```
run schema.sql
if [ "${SEED_SIZE:-100}" = "1000" ]; then
  run seed_1000.sql
else
  run seed.sql
fi
```

What's happening: sqlcmd over mssql-tools18 with a 60×5s wait loop; run() fails fast on any script error. SEED_SIZE selects the 100- or 1000-product catalog; seed_product_images.sql always runs last as idempotent UPDATEs pointing ImageUrls at local /Images/products photos.

Why it matters: Startup ordering is handled in shell rather than compose guesswork: web only starts after dbinit succeeds. Photo fixup is a no-op for unknown slugs, so re-runs and catalog growth never break.

Byte 3: Deterministic catalogs: gen_seed.py and gen_seed1500.py

Builds on: Bytes 1–2

In plain terms:

Python generators turn small curated name lists into hundreds of priced, stocked, photographed products with size variants — deterministically, so every environment gets the same demo shop.

The code:

```
python
h=int(hashlib.md5(name.encode()).hexdigest()[:8],16)
price=round(149 + (h%4850) + ((h%100)/100.0),2)
cmp_price=round(price*1.25,2) if idx%3==0 else None
s=slug(name) if n_target<=200 else slug(name)+("-%d"%idx)
```

What's happening: gen_seed.py builds the 100- and 1000-product seeds across 18 categories (8 roots + 10 children) with MD5-derived prices/stock, every third item discounted, featured/new flags on cadences, Size variants for fashion/footwear, and a NAME_IMG map from product names to local photo files. gen_seed1500.py adds 1500 MASS-namespace products with per-category images and size sets. Every INSERT is guarded by IF NOT EXISTS on Slug; seed_product_images.sql maps slugs to the checked-in product photos.

Why it matters: Determinism means screenshots, tests and reviews agree everywhere. Slug suffixing past 200 products avoids collisions; IF NOT EXISTS guards make seeds re-runnable; the photo map is why grids look photo-diverse instead of placeholder-grey.

PUTTING IT TOGETHER

Schema first (idempotent tables, unique slugs/keys, cascading children), then data (deterministic seeds sized by SEED_SIZE), then photos (idempotent image fixups) — all sequenced by a wait-loop init script. The database that results matches the EF mapping 1:1, which is why the app can treat the model as a faithful mirror of SQL Server.

13 Operations — compose, containers, proxy, config

These are YAML, Dockerfile, shell, Nginx and XML config files that build, wire and serve the app. No secret values are reproduced here — key names and behaviours only. Files: docker-compose.yml, docker/Dockerfile.web, docker/web-entrypoint.sh, nginx/nginx.conf, nginx/maintenance.html, Ecommerce.Web/Web.config, .env.example, .dockerignore, .gitignore.

Byte 1: Four containers, one network: docker-compose.yml

Builds on: None — starting point

In plain terms:

An isolated stack — database, one-shot init, web app, reverse proxy — on its own network and volume, with the outside world reaching only Nginx on host port 8085.

The code:

```yaml
services:
  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
  dbinit:
    condition: service_completed_successfully  # gates web start
  web:
    expose: ["8080"]  # internal only
  nginx:
    ports: ["8085:80"]
```

What's happening: db persists to a named volume with a sqlcmd health check; dbinit mounts schema/seed/init read-only and exits after success; web builds from docker/Dockerfile.web, takes DB_* plus optional ADMIN_* from the environment, and health-checks /health; nginx waits for a healthy web and publishes 8085. Depends-on conditions (healthy / completed_successfully) enforce start order.

Why it matters: Isolation (own network, volume and project name) means this stack cannot disturb sibling projects. The compose file requires DB_PASSWORD from .env and fails fast with a clear message when it is missing.

Byte 2: Build and boot on Mono: Dockerfile.web and web-entrypoint.sh

Builds on: Byte 1

In plain terms:

The image compiles the .NET Framework solution with Mono's MSBuild (no host toolchain needed), ships only the web output plus exact MVC/Razor binaries, then at boot injects the real connection string and starts XSP4.

The code:

```dockerfile
RUN mono /usr/local/bin/nuget.exe restore LegacyEcommerce.sln \
    && msbuild /p:Configuration=Release ... LegacyEcommerce.sln
CS="Server=${DB_HOST},${DB_PORT};Database=${DB_NAME};..."
exec xsp4 --port 8080 --address 0.0.0.0 --root /app --nonstop
```

What's happening: Multi-stage: builder restores NuGet and builds Release; runtime installs mono-xsp4/curl, copies the site, and pins System.Web.Mvc/Razor/WebPages to the exact NuGet 5.2.7/3.2.7 binaries (guarding

against Mono GAC 2.0 copies). Buster apt sources are repointed to the archive (verified fix). The entrypoint refuses to start without DB_PASSWORD and sed-replaces the __DB_PASSWORD__ placeholder in Web.config.

Why it matters: Hosts stay clean (no .NET/Mono installs); runtime stays lean (no build tools). Entrypoint injection keeps the real password out of the image and lets one image serve any environment.

Byte 3: Front door: nginx.conf and maintenance.html

Builds on: Bytes 1–2

In plain terms:

Nginx terminates public traffic, caches static assets briefly, preserves host and port so app redirects stay reachable, and shows a friendly page when the app is down — never container internals.

The code:

```nginx
upstream web_app { server web:8080; }
proxy_set_header Host $http_host;  # preserves localhost:8085 for redirects
proxy_intercept_errors on;
error_page 502 503 504 /maintenance.html;
```

What's happening: Asset locations proxy to XSP with 1-hour public caching; / proxies with forwarded headers and cookie-path/redirect handling; 502/503/504 render the internal-only maintenance page; /nginx-health answers the compose probe. $http_host (not $host) is the deliberate choice so ReturnUrl and OWIN redirects keep the original port.

Why it matters: Single public port, graceful degradation, and no leaked topology. Static caching is short on purpose — product photos change with seeds.

Byte 4: App settings without secrets: Web.config and .env.example

Builds on: Bytes 1–3

In plain terms:

Web.config carries a placeholder connection string plus framework settings; the real password arrives only via environment at container start. The example env file documents key names and never holds real credentials.

The code:

```xml
<add name="EcommerceDb" connectionString="Server=db,1433;...Password=__DB_PASSWORD__;..." />
<sessionState mode="InProc" timeout="60" />
<customErrors mode="RemoteOnly" defaultRedirect="~/Home/Error" />
# .env.example keys: DB_PASSWORD (required), SEED_SIZE (default 100), ADMIN_EMAIL / ADMIN_PASSWORD
(optional, local/dev only)
```

What's happening: Notable settings: Entity Framework SQL provider, InProc 60-minute session, RemoteOnly errors (local stack traces, remote friendly page), en-IN culture, extensionless MVC routing, and assembly binding redirects pinning MVC 5.2.7 et al. .env.example instructs copying to .env (git-ignored) with a strong password; admin keys stay blank on public deployments.

Why it matters: Secrets live in the environment, never in git or the image. Operators get one documented file to copy; reviewers can audit every behaviour here without seeing a single credential.

PUTTING IT TOGETHER

Compose isolates and orders four containers; the Mono image builds without host tooling and boots only when secrets are present; Nginx publishes one friendly port with graceful errors; configs separate code (committed) from secrets (environment-only). The result runs the same from a laptop demo to any Docker host with one password set.

14 Tests and operator notes

These are PowerShell HTTP-level scripts plus a shell helper and a runbook text file. They drive the running site like a user would — no unit-test framework involved. Files: tests/smoke.ps1, tests/features.ps1, tests/extended.ps1, tests/retest.sh, docs/start_and_stopping.txt.

Byte 1: Smoke: anonymous routes, cart, guest checkout, auth loop

Builds on: None — starting point

In plain terms:

The deliverable test script walks the golden path end to end: health, home, catalog, category, product, AJAX grid, add-to-cart, guest checkout through to the success page, then register/profile/logoff/login.

The code:

```
powershell
$r = Get-Page (New-Sess) "/health"; Assert "health" ($r.Content -eq "OK")
$add = Post-NoRedir $s "/Cart/Add" @{ __RequestVerificationToken=$tok; productId="1"; quantity="2" }
$c = Get-Page $s "/Cart/Count"; Assert "cart-count-2" ($c.Content -match '"count":2')
```

What's happening: Helpers keep a cookie session (New-Sess), scrape the anti-forgery token with a regex, and POST without following redirects to assert 302 targets. Each step prints PASS/FAIL and the script exits with the failure count. Run: powershell -ExecutionPolicy Bypass -File tests/smoke.ps1 (optional -Base override).

Why it matters: One command proves the deployment serves, sells and signs up. Token scraping mirrors exactly what a browser does, so anti-forgery regressions fail loudly here first.

Byte 2: Features, extended and helpers

Builds on: Byte 1

In plain terms:

Beyond smoke: pagination, search/sort, infinite scroll JSON, cart AJAX shapes, idempotent confirm (same key, one order), admin authorisation, bad-login rejection, order-in-history, asset-reference resolution, and data persistence across restarts.

The code:

```
powershell
# features.ps1 covers: pagination, search/sort, infinite scroll, cart AJAX,
# confirmation + idempotent confirm, and admin authorisation.
# extended.ps1: bad login, authed order history, asset refs, persistence.
```

What's happening: features.ps1 accepts optional admin credentials for local runs (never committed); extended.ps1 registers a fresh user, places an order, and asserts it appears in /Account/Orders, then crawls homepage asset references and requests each one. retest.sh clears the Mono ASP.NET compile cache, touches Web.config to recycle the app, and probes a route. docs/start_and_stopping.txt is the copy-paste runbook: config, build, start, health, logs, tests, stop/restart, volume location, port-conflict check — with the destructive reset explicitly marked DANGER/never-auto-run.

Why it matters: Coverage targets behaviour, not lines: double-submit safety, auth gating and link rot are all asserted against the live stack. The runbook keeps operations boring and reversible (stop keeps data; only the documented danger commands drop the volume).

PUTTING IT TOGETHER

Smoke proves the path, features prove the edges (paging, scroll, idempotency, admin gates), extended proves integrity (history, assets, persistence), and the runbook makes every lifecycle step a paste away. Together they are the project's executable specification: if these pass against http://localhost:8085, the store is demonstrably working.

15 Coverage summary and verification

Files examined and documented

Group	Files documented	Count
Solution/readme/ops docs	LegacyEcommerce.sln, README.md, docs/start_and_stopping.txt, .gitignore, .dockerignore	5
Core	Entities.cs, Interfaces.cs, Common.cs, ViewModels.cs, AdminViewModels.cs	5
Data	EcommerceDbContext.cs, Repositories.cs, DatabaseHealthCheck.cs (+ EcommerceModel.edmx as design artefact)	3+1
Services	Mapping.cs, CatalogService.cs, CartService.cs, CheckoutAccountServices.cs, AdminService.cs	5
Web composition	Global.asax, Global.asax.cs, Startup.cs, RouteConfig.cs, UnityConfig.cs, BundleConfig.cs, FilterConfig.cs, UnityResolver.cs, CartSession.cs, AdminBootstrap.cs, IdentityModels.cs	11
Controllers	Home, Catalog, Cart, Checkout, Account, Admin	6
Views	~30 .cshtml incl. layout, header/footer, catalog, product, cart, checkout, account, admin, static, partials	~30
Client	site.js, site.css	2
Database	schema.sql, init.sh, gen_seed.py, gen_seed1500.py, seed.sql, seed_1000.sql, seed_1500.sql, seed_product_images.sql	8
Ops config	docker-compose.yml, Dockerfile.web, web-	7

	entrypoint.sh, nginx.conf, maintenance.html, Web.config, .env.example	
Tests	smoke.ps1, features.ps1, extended.ps1, retest.sh	4

Total: approximately 85 source/config files examined and documented (about 250 binary product/category images counted as one asset group, not individually).

Files skipped with reasons

- _backup_*/ (8 directories: _backup_minimal, _backup_pre_continuation_20261011_155831, _backup_stabilize, _backup_ui_20261011, _backup_ui_compact, _backup_ui_coral, _backup_ui_dash, _backup_ui_redesign2/) — superseded snapshots, excluded by .gitignore/.dockerignore; purpose noted, contents not documented.
- .git/ internals — version-control metadata, not application code.
- .env — present locally but never read; secret values excluded by requirement (only key names from .env.example are described).
- Ecommerce.Web/Images/** (~250 .png/.jpg) — binary assets; role (category icons, product photos, placeholder fallback) documented, bytes not transcribed.
- db/seed_1000.sql, db/seed_1500.sql, db/seed.sql (generated, tens of thousands of lines each) — characterised via their generators; re-run safety and structure documented rather than quoted.
- packages.config / *.csproj / AssemblyInfo.cs — package manifests and build metadata; stack versions noted (MVC 5.2.7, EF6, .NET 4.7) without line-by-line treatment.
- Vendor copies (bootstrap.min.css/js, jquery*.js, validation scripts) — third-party libraries, used but not authored here.
- Ecommerce.Data/EcommerceModel.edmx — retained design artefact; runtime mapping lives in EcommerceDbContext.cs (documented).
- db/mkvxy.py — one-off experiment scaffolding header variants; superseded by the committed header, noted only.

Verification

- Destination: D:\Yantra3loka LLP\Training\Week 3 (5 October to 11 October)\Knowledge Bytes.docx
- Filename rule: no pre-existing Knowledge Bytes*.docx was found, so the exact required name Knowledge Bytes.docx is used.
- Secrets: .env was never opened; no passwords, tokens or keys appear in this document (.env.example values described by key name only).
- Source integrity: this task performed reads only; no application file was created, modified, renamed or deleted (verified: generation ran from a temp script outside the project).
- Fidelity: snippets are copied from the actual files read above; behaviours described (routes, tariffs, metric rules, token sharing, idempotency) match the code; limits are stated explicitly (generated SQL characterised, images grouped, edmx/mkvxy noted).
- Format: cover page, field-based table of contents (right-click → Update Field in Word), heading hierarchy, footer page numbers, 1in/0.9in margins, shaded Consolas code blocks, tables for the file map and coverage, page breaks between major sections.

To validate after opening in Word: update the TOC field, confirm page numbers appear in the footer, skim one code block per section for monospace shading, and search for password-like strings (none should exist beyond the __DB_PASSWORD__ placeholder token).